A. Requirements Specification

1. Problem Statement and Project Goals The objective of this project is to develop a robust, maintainable E-Commerce Management System. The system aims to simulate a real-world online store where customers can browse a product catalog, manage their shopping carts, and place orders. The core architectural goal is to apply Object-Oriented Programming (OOP) principles and specifically implement three design patterns: Strategy, Decorator, and Adapter, to handle dynamic business rules such as pricing, order customization, and external payment integration.

2. Target Users and Their Roles

- **Customer:** A user who interacts with the storefront. Can view the product catalog, filter items, add/remove items from the shopping cart, apply discounts, choose delivery options, and complete purchases using a simulated payment gateway.
- **Administrator (Admin):** A privileged user responsible for store operations. Can manage the product catalog (add,update, delete products), manage stock quantities, and view the status of all customer orders.

3. Functional Requirements

- **Product Management:** The system must allow admins to add, update, and remove products, including defining categories, descriptions, prices, and stock levels.
- **Browsing and Search:** Customers must be able to view all available products and search/filter them by category.
- **Cart Management:** Customers must be able to add products to a shopping cart, modify quantities, and remove items.
- **Checkout and Order Creation:** The system must convert a cart into an order, updating the inventory accordingly.
- **Discount Application (Strategy Pattern):** Customers can select different promotional strategies (e.g., Percentage Discount, Fixed Amount Discount) that dynamically calculate the final order price.
- **Order Customization (Decorator Pattern):** Customers must be able to add optional services to their order dynamically, such as Express Delivery or Premium Gift Wrapping, which alter the final price and order description.
- **Payment Processing (Adapter Pattern):** The system must process payments by integrating with a simulated external payment API (e.g., Third-Party Bank Gateway) using an adapter.
- **Order Tracking:** Customers and admins must be able to view order history and track order statuses (e.g., Pending,Paid, Shipped).

4. Non-Functional Requirements

- **Data Persistence:** All entities (Users, Products, Orders) must be stored in a relational database (PostgreSQL) to ensure data is not lost between application restarts.

- **Maintainability:** The architecture must enforce a strict separation of concerns (Presentation, Business Logic, Data Access layers).
- **Validation:** The system must handle invalid inputs gracefully (e.g., negative product quantities, insufficient stock during checkout) without crashing.
- **User Interface:** The system will provide an interactive text-based Console Interface (or basic Web UI) for user interaction.

